QUIZ Pertemuan 3 — Encapsulation pada Python

Nama	**:**	**Jihan Maharani**
NIM	**:**	**250660221031**
Kelas	**:**	**Sistem Informasi 3-A**
Mata Kuliah	**:**	**Pemrograman Berorientasi Objek**

SOAL

1. **Soal 1 — Tujuan Encapsulation**
   Jelaskan tujuan encapsulation dalam OOP. Mengapa data tertentu perlu dikendalikan melalui method atau property?

   Jawaban:
   Encapsulation adalah salah satu konsep OOP yang digunakan untuk melindungi data dalam sebuah class agar tidak dapat diubah secara sembarangan dari luar class. Tujuannya adalah menjaga keamanan dan konsistensi data dengan cara mengatur akses melalui method atau property.
   Data tertentu perlu dikendalikan karena perubahan tanpa aturan dapat menyebabkan kesalahan pada program.
   Contohnya, saldo rekening tidak boleh diubah menjadi nilai negatif. Oleh karena itu, perubahan saldo harus melalui method atau property yang memiliki validasi.

2. **Soal 2 — Private Attribute dan Property**
   Jelaskan fungsi nama attribute dengan awalan __ dan peran decorator @property serta @<nama>.setter.

   Jawaban:
   1. Private Attribute (__)
   Private attribute adalah atribut yang menggunakan awalan dua garis bawah (__) untuk membatasi akses langsung dari luar class.
   Contoh:

```
class Mahasiswa:
    def __init__(self, nama):
        self.__nama = nama
```

   Atribut __nama tidak dapat diakses secara langsung menggunakan objek.__nama karena Python menerapkan name mangling.

   2. @property
   Decorator @property digunakan agar atribut dapat dibaca seperti atribut biasa, tetapi sebenarnya melalui sebuah method.

Contoh:
@property
def nama(self):
  return self.__nama

3. @nama.setter
Decorator @nama.setter digunakan untuk mengatur atau mengubah nilai atribut melalui method yang dapat dilengkapi validasi.
Contoh:
@nama.setter
def nama(self, nilai):
  self.__nama = nilai

Jadi, @property digunakan untuk membaca nilai, sedangkan @setter digunakan untuk mengubah nilai dengan aturan tertentu.

3. Soal 3 — Prediksi Output

```
class Mahasiswa:
    def __init__(self, nama, sks):
        self.nama = nama
        self.__sks = sks

    @property
     def sks(self):
         return self.__sks

mhs = Mahasiswa("Jhon Doe", 3)
print(mhs.sks)
```

Tuliskan output dan jelaskan bagaimana property membaca private attribute.

Jawaban:

Output yang dihasilkan:

3

Penjelasan:

Pada kode tersebut:

self.__sks = sks

Atribut __sks merupakan private attribute yang menyimpan jumlah SKS mahasiswa.

Kemudian terdapat:

@property

def sks(self):

    return self.__sks

Property sks digunakan untuk membaca nilai dari private attribute __sks.

Ketika perintah berikut dijalankan:

print(mhs.sks)

Python memanggil method property sks dan mengembalikan nilai __sks, yaitu 3.

Dengan demikian, private attribute tetap dapat dibaca melalui property tanpa harus mengaksesnya secara langsung.

4. Soal 4 — Validasi Setter

Lengkapi setter berikut agar nilai saldo negatif menghasilkan ValueError.

class Rekening:

    def __init__(self, saldo):

        self.__saldo = 0

        self.saldo = saldo

    @property

    def saldo(self):

        pass

    @saldo.setter

    def saldo(self, nilai):

        pass

Jawaban:

Kode yang sudah dilengkapi:

```
class Rekening:
    def __init__(self, saldo):
        self.__saldo = 0
        self.saldo = saldo

    @property
    def saldo(self):
        return self.__saldo

    @saldo.setter
    def saldo(self, nilai):
        if nilai < 0:
            raise ValueError("Saldo tidak boleh negatif.")
        self.__saldo = nilai



# Pengujian program
rekening1 = Rekening(500000)

print("Saldo:", rekening1.saldo)

rekening1.saldo = 750000
print("Saldo setelah diubah:", rekening1.saldo)
```

Output:

Saldo: 500000

Saldo setelah diubah: 750000

Jika saldo diubah menjadi negatif:

rekening1.saldo = -100000

Maka program menghasilkan error:

ValueError: Saldo tidak boleh negatif.

Penjelasan:

- self.__saldo = 0 memberikan nilai awal private attribute.
- @property digunakan untuk membaca saldo.
- @saldo.setter digunakan untuk mengubah saldo.
- if nilai < 0 memeriksa apakah saldo bernilai negatif.
- raise ValueError menghasilkan error jika saldo tidak memenuhi aturan.
- self.__saldo = nilai menyimpan saldo jika nilainya valid.

5. Soal 5 — Analisis Access Control

Apa risiko jika attribute saldo dapat diubah langsung tanpa validasi? Berikan satu contoh aturan validasi yang sesuai untuk rekening mahasiswa.

Jawaban:

Jika attribute saldo dapat diubah secara langsung tanpa validasi, terdapat risiko data menjadi tidak konsisten dan tidak sesuai dengan aturan rekening.

Contohnya, seseorang dapat mengubah saldo menjadi nilai negatif atau memasukkan nilai yang tidak sesuai dengan ketentuan sistem.

Salah satu contoh aturan validasi untuk rekening mahasiswa adalah saldo tidak boleh bernilai negatif.

Contoh penerapan:

```
@saldo.setter
def saldo(self, nilai):
    if nilai < 0:
        raise ValueError("Saldo tidak boleh negatif.")
    self.__saldo = nilai
```

Dengan adanya validasi tersebut, sistem dapat memastikan bahwa perubahan saldo selalu mengikuti aturan yang telah ditentukan.